

Análisis de Algoritmos

Mecánica de la recursión: pila de llamadas, traza e invariantes

Luis Alfredo Alvarado Rodríguez

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
ESCUELA DE CIENCIAS FÍSICAS Y MATEMÁTICAS

26 de agosto de 2025

Sumario

- 1 Recursión: fundamentos
- 2 Ordenación: de doble bucle a Quicksort
- 3 Conclusiones

Mapa de la sección

- 1 Recursión: fundamentos
 - Definiciones fundamentales
 - Introducción
 - Motivación
 - Factorial: iterativo vs recursivo
 - Pila y traza
 - Plantilla general
- 2 Ordenación: de doble bucle a Quicksort
- 3 Conclusiones

Definiciones fundamentales

Definición 1 (Recursión)

Técnica de diseño de algoritmos donde una función o procedimiento se define en términos de sí mismo, resolviendo un problema mediante la solución de instancias más pequeñas del mismo, hasta alcanzar un caso base que no requiere más subdivisión.

Definiciones fundamentales

Definición 1 (Recursión)

Técnica de diseño de algoritmos donde una función o procedimiento se define en términos de sí mismo, resolviendo un problema mediante la solución de instancias más pequeñas del mismo, hasta alcanzar un caso base que no requiere más subdivisión.

Definición 2 (Pila de llamadas)

Estructura de datos de tipo LIFO (last-in, first-out) que gestiona las invocaciones activas de funciones o procedimientos durante la ejecución de un programa, almacenando parámetros, variables locales y la dirección de retorno de cada llamada.

Definición 3 (Traza)

Secuencia registrada (o reconstruida) de entradas y salidas de funciones/procedimientos durante la ejecución de un algoritmo, que muestra explícitamente el orden de invocación y de retorno, reflejando la profundidad en la pila de llamadas.

Definiciones fundamentales

Definición 3 (Traza)

Secuencia registrada (o reconstruida) de entradas y salidas de funciones/procedimientos durante la ejecución de un algoritmo, que muestra explícitamente el orden de invocación y de retorno, reflejando la profundidad en la pila de llamadas.

Definición 4 (Invariante)

Propiedad lógica que se mantiene verdadera en todo el transcurso de la ejecución de un algoritmo, ya sea en cada iteración de un bucle o en cada llamada recursiva, y que sirve para demostrar formalmente la corrección del procedimiento.

Introducción: recursión e inducción

Dos maneras de definir $n!$

- (1) “ $n!$ es el producto de todos los números enteros desde 1 hasta n , inclusive.”
- (2) “Si $n = 1$ entonces $n! = 1$, de lo contrario $n! = n \cdot (n - 1)!$.”

Introducción: recursión e inducción

Dos maneras de definir $n!$

- (1) “ $n!$ es el producto de todos los números enteros desde 1 hasta n , inclusive.”
- (2) “Si $n = 1$ entonces $n! = 1$, de lo contrario $n! = n \cdot (n - 1)!$.”

Validez de la definición recursiva

- La definición (2) refiere a la función en un *argumento menor* $(n - 1)$.
- La corrección descansa en **inducción matemática**: la función queda bien definida para todo $n \in \mathbb{Z}_{>0}$.

¿Por qué pensar recursivamente?

- Lenguajes de alto nivel que **admiten recursión**: Pascal, PL/C, Lisp, APL, C, entre otros.

¿Por qué pensar recursivamente?

- Lenguajes de alto nivel que **admiten recursión**: Pascal, PL/C, Lisp, APL, C, entre otros.
- **Sin bucles visibles**: el compilador gestiona la *pila de llamadas* y el “mantenimiento de registros”.

¿Por qué pensar recursivamente?

- Lenguajes de alto nivel que **admiten recursión**: Pascal, PL/C, Lisp, APL, C, entre otros.
- **Sin bucles visibles**: el compilador gestiona la *pila de llamadas* y el “mantenimiento de registros”.
- La recursión permite expresar ideas con naturalidad; muchos métodos potentes se formulan recursivamente.

¿Por qué pensar recursivamente?

- Lenguajes de alto nivel que **admiten recursión**: Pascal, PL/C, Lisp, APL, C, entre otros.
- **Sin bucles visibles**: el compilador gestiona la *pila de llamadas* y el “mantenimiento de registros”.
- La recursión permite expresar ideas con naturalidad; muchos métodos potentes se formulan recursivamente.
- El **caso trivial** (base) es esencial: detiene la ejecución y dispara el retorno.

Módulo *no* recursivo para $n!$

```
function fact(n);  
{calcula  $n!$  para  $n > 0$ }  
fact := 1;  
for i := 1 to n do fact := i * fact;  
end.
```

Módulo *no* recursivo para $n!$

```
function fact(n);  
{calcula  $n!$  para  $n > 0$ }  
fact := 1;  
for i := 1 to n do fact := i * fact;  
end.
```

Observación

Hay un **bucle visible** que acumula el producto; el estado (i, fact) vive en el mismo marco de activación.

Módulo recursivo para $n!$

```
function fact(n);  
if n = 1 then fact := 1  
else fact := n * fact(n - 1);  
end.
```


Módulo recursivo para $n!$

```
function fact(n);  
if n = 1 then fact := 1  
else fact := n * fact(n - 1);  
end.
```

Caso base

El caso $n = 1$ **detiene** la recursión. Sin este caso, el programa descendería indefinidamente.

Módulo recursivo para $n!$

```
function fact(n);  
if n = 1 then fact := 1  
else fact := n * fact(n - 1);  
end.
```

Caso base

El caso $n = 1$ **detiene** la recursión. Sin este caso, el programa descendería indefinidamente.

Idea clave

Cada llamada usa un **problema más pequeño** ($n - 1$) y el resultado se combina al regresar.

Pila de llamadas: `fact(5)`

Descenso (invocaciones)

- `fact(5) → fact(4) → fact(3) → fact(2) → fact(1)`

Pila de llamadas: `fact(5)`

Descenso (invocaciones)

■ `fact(5) → fact(4) → fact(3) → fact(2) → fact(1)`

Ascenso (retornos)

`fact(1) = 1,   fact(2) = 2 · 1 = 2,   fact(3) = 3 · 2 = 6`

`fact(4) = 4 · 6 = 24,   fact(5) = 5 · 24 = 120`

Pila de llamadas: `fact(5)`

Descenso (invocaciones)

■ `fact(5) → fact(4) → fact(3) → fact(2) → fact(1)`

Ascenso (retornos)

`fact(1) = 1,   fact(2) = 2 · 1 = 2,   fact(3) = 3 · 2 = 6`

`fact(4) = 4 · 6 = 24,   fact(5) = 5 · 24 = 120`

Interpretación

Cada llamada crea un **marco** en la pila con sus parámetros/locales y la *dirección de retorno*.

Traza de ejecución (entradas/salidas)

```
enter fact(n=5)
  enter fact(n=4)
    enter fact(n=3)
      enter fact(n=2)
        enter fact(n=1)
          leave fact(n=1) -> 1
        leave fact(n=2) -> 2
      leave fact(n=3) -> 6
    leave fact(n=4) -> 24
  leave fact(n=5) -> 120
```

Traza de ejecución (entradas/salidas)

```
enter fact(n=5)
  enter fact(n=4)
    enter fact(n=3)
      enter fact(n=2)
        enter fact(n=1)
          leave fact(n=1) -> 1
        leave fact(n=2) -> 2
      leave fact(n=3) -> 6
    leave fact(n=4) -> 24
  leave fact(n=5) -> 120
```

Lectura de la traza

- La **indentación** refleja la *profundidad* en la pila.
- Las **salidas** ocurren en orden inverso a las entradas (LIFO).

Plantilla de una rutina recursiva

Estructura global

1. **Si** caso trivial: hacer cosa trivial.

Plantilla de una rutina recursiva

Estructura global

1. **Si** caso trivial: hacer cosa trivial.
2. **Si no**: llamar al mismo procedimiento con *valores más pequeños*.

Plantilla de una rutina recursiva

Estructura global

1. **Si** caso trivial: hacer cosa trivial.
2. **Si no**: llamar al mismo procedimiento con *valores más pequeños*.
3. (Opcional) combinar resultados al regresar.

Checklist

- ¿Hay **caso base** que siempre se alcanza?
- ¿La llamada recursiva **disminuye** una medida (tamaño, índice, etc.)?

Mapa de la sección

- 1 Recursión: fundamentos
- 2 Ordenación: de doble bucle a Quicksort
 - Planteamiento
 - **slowsort** (referencia iterativa)
 - Quicksort: invariante y recurrencia
- 3 Conclusiones

Ordenar por comparaciones e intercambios

Problema

Dado un arreglo $x[1], \dots, x[n]$, reordenarlo en orden **no decreciente**.

Ordenar por comparaciones e intercambios

Problema

Dado un arreglo $x[1], \dots, x[n]$, reordenarlo en orden **no decreciente**.

Ejemplo

Entrada: $\{9, 4, 7, 2, 1\}$ $\Rightarrow$ Salida: $\{1, 2, 4, 7, 9\}$.

Ordenar por comparaciones e intercambios

Problema

Dado un arreglo $x[1], \dots, x[n]$, reordenarlo en orden **no decreciente**.

Ejemplo

Entrada: $\{9, 4, 7, 2, 1\}$ $\Rightarrow$ Salida: $\{1, 2, 4, 7, 9\}$.

Operaciones permitidas

- Comparar dos claves del arreglo.
- Intercambiar posiciones de dos claves.

slowsort: esquema de doble bucle

```
procedimiento slowsort(X: arreglo[1..n])  
  // ordena X en orden no decreciente (intención)  
  for j := 1 to n-1 do  
    for r := j+1 to n do  
      if x[j] < x[r] then  
        intercambiar(x[j], x[r])
```

slowsort: esquema de doble bucle

```
procedimiento slowsort(X: arreglo[1..n])  
  // ordena X en orden no decreciente (intención)  
  for j := 1 to n-1 do  
    for r := j+1 to n do  
      if x[j] < x[r] then  
        intercambiar(x[j], x[r])
```

Invariante de bucle

- Al terminar el ciclo interno, $x[j]$ es el **máximo** de $x[j..n]$.
- Tras la j -ésima iteración externa, las posiciones $1..j$ contienen los j *mayores* en orden no creciente.

slowsort: esquema de doble bucle

```
procedimiento slowsort(X: arreglo[1..n])  
  // ordena X en orden no decreciente (intención)  
  for j := 1 to n-1 do  
    for r := j+1 to n do  
      if x[j] < x[r] then  
        intercambiar(x[j], x[r])
```

Invariante de bucle

- Al terminar el ciclo interno, $x[j]$ es el **máximo** de $x[j..n]$.
- Tras la j -ésima iteración externa, las posiciones $1..j$ contienen los j *mayores* en orden no creciente.

Cuidado con el sentido de comparación

Con `if x[j] < x[r] then swap` el resultado queda **no creciente**.
Para orden *no decreciente*, invierta la desigualdad o el intercambio.

Invariantes en recursión: esquema de Quicksort

Partición (invariante central)

Dados arreglo A y pivote p :

- Para todo $i \leq p - 1$: $A[i] \leq \text{pivote}$.
- Para todo $i \geq p + 1$: $A[i] \geq \text{pivote}$.

Invariantes en recursión: esquema de Quicksort

Partición (invariante central)

Dados arreglo A y pivote p :

- Para todo $i \leq p - 1$: $A[i] \leq \text{pivote}$.
- Para todo $i \geq p + 1$: $A[i] \geq \text{pivote}$.

Recurrencia y progreso

- Subproblemas: ordenar $A[1..p - 1]$ y $A[p + 1..n]$.

Invariantes en recursión: esquema de Quicksort

Partición (invariante central)

Dados arreglo A y pivote p :

- Para todo $i \leq p - 1$: $A[i] \leq \text{pivote}$.
- Para todo $i \geq p + 1$: $A[i] \geq \text{pivote}$.

Recurrencia y progreso

- Subproblemas: ordenar $A[1..p - 1]$ y $A[p + 1..n]$.
- **Caso base**: segmentos de tamaño 0 o 1 ya están ordenados.

Invariantes en recursión: esquema de Quicksort

Partición (invariante central)

Dados arreglo A y pivote p :

- Para todo $i \leq p - 1$: $A[i] \leq \text{pivote}$.
- Para todo $i \geq p + 1$: $A[i] \geq \text{pivote}$.

Recurrencia y progreso

- Subproblemas: ordenar $A[1..p - 1]$ y $A[p + 1..n]$.
- **Caso base**: segmentos de tamaño 0 o 1 ya están ordenados.
- **Medida decreciente**: $|A[1..p - 1]|, |A[p + 1..n]| < n$.

Mapa de la sección

- 1 Recursión: fundamentos
- 2 Ordenación: de doble bucle a Quicksort
- 3 Conclusiones**

- La recursión se valida por **inducción** y se sostiene en dos pilares: **caso base** y **medida decreciente**.

- La recursión se valida por **inducción** y se sostiene en dos pilares: **caso base** y **medida decreciente**.
- La **pila de llamadas** organiza marcos (parámetros, locales, retorno); la **traza** revela el orden LIFO.

- La recursión se valida por **inducción** y se sostiene en dos pilares: **caso base** y **medida decreciente**.
- La **pila de llamadas** organiza marcos (parámetros, locales, retorno); la **traza** revela el orden LIFO.
- Los **invariantes** explican la corrección:
 - En iteración (p. ej., **sort**): qué propiedad conserva cada ciclo.
 - En recursión (p. ej., Quicksort): partición correcta + subproblemas más pequeños.

- La recursión se valida por **inducción** y se sostiene en dos pilares: **caso base** y **medida decreciente**.
- La **pila de llamadas** organiza marcos (parámetros, locales, retorno); la **traza** revela el orden LIFO.
- Los **invariantes** explican la corrección:
 - En iteración (p. ej., **sort**): qué propiedad conserva cada ciclo.
 - En recursión (p. ej., Quicksort): partición correcta + subproblemas más pequeños.
- Diseñar recursión = elegir buenas variables y *casos* para que la solución emerja de combinar subproblemas.